Time-Series Database Engine

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a small time-series database (TSDB) that ingests timestamped numeric measurements, stores them on disk in a heavily compressed binary format, and answers range queries over them. The result is a network server that listens on a TCP port, speaks a tiny text protocol, and can be exercised from a simple command-line client.

Time-series data, CPU usage every second, stock prices every tick, sensor readings every minute, has properties that distinguish it from the data you stored in a general-purpose database in Project 01. First, writes are almost always append-only: new points arrive with timestamps greater than any previously stored point. Second, the timestamps themselves are highly regular, often spaced at fixed intervals. Third, consecutive values within the same metric tend to be very similar to each other (CPU usage at 45.2% is usually followed by CPU usage at 45.3%, not 99.9%). Your TSDB will exploit all three properties.

The two compression techniques at the heart of this project are the ones Facebook published in 2015 in the Gorilla paper, and the same ones that power Prometheus, InfluxDB, and every modern monitoring system. The first is *delta-of-delta encoding* for timestamps: instead of storing each absolute timestamp, you store the difference between successive deltas, which is almost always zero for regularly-sampled metrics. The second is *XOR-based encoding* for floating-point values: the IEEE 754 binary representations of two close numbers usually differ in only a few bits, and storing just those bits compresses the value column dramatically. Together, these two techniques reduce a naive 16-byte-per-point format to roughly 1.4 bytes per point on real workloads, a tenfold reduction, while still allowing scans at hundreds of thousands of points per second.

The compression schemes are bit-level: you will be reading and writing individual bits into byte buffers, handling sub-byte alignment, and tracking your position in a bitstream. Debugging bit-packing bugs is unpleasant because the symptoms are usually "the wrong number came back" with no obvious cause. Plan time for testing each encoder and decoder in isolation *before* wiring them into the storage engine. The required reading at the end of this manual includes the original Gorilla paper, which you should read before Phase 2; it is short and clear and will save you considerable time.

You are not building a general-purpose database. There is no SQL, no labels or tags on metrics (each metric is identified by a flat name like `cpu.usage`, multi-dimensional metrics are out of scope), no joins, no transactions, and no concurrency control beyond what is needed to serve multiple clients. Your effort goes into four things: ingesting points from the network, buffering them in memory, flushing them to disk in the compressed binary format, and answering range queries that read from both memory and disk.

2 What the Final Product Looks Like

When your project is complete, the server can be started from the command line, and a separate client can push data into it and query it back.

```
$ make
$ ./tsdb --data ./data --port 5555
tsdb started on port 5555, data directory ./data

[in another terminal]
$ ./tsdb-cli localhost 5555
connected to tsdb at localhost:5555

> PUT cpu.usage 1728000000 45.2
OK
> PUT cpu.usage 1728000010 45.8
OK
```

```
> PUT cpu.usage 1728000020 46.1
OK
> PUT cpu.usage 1728000030 47.0
OK

> GET cpu.usage 1728000000 1728000040
1728000000 45.2
1728000010 45.8
1728000020 46.1
1728000030 47.0
(4 points)

> AGG cpu.usage 1728000000 1728000040 20 avg
1728000000-1728000020 45.5
1728000020-1728000040 46.55
(2 buckets)

> STATS cpu.usage
metric:           cpu.usage
total points:     4
in memory:        4
on disk:          0
disk chunks:      0
first timestamp:  1728000000
last timestamp:   1728000030

> FLUSH cpu.usage
Flushed 4 points to disk: 47 bytes (header 32 + ts 7 + values 8)
  vs naive 16 bytes/point = 64 bytes
  compression ratio: 1.36x  (small chunk, ratio improves with size)

> QUIT
```

The benchmark in Phase 3 will load 500,000 points across many metrics and measure two things: ingestion throughput, and disk size compared to the naive 16-bytes-per-point baseline. A successful submission sustains at least 50,000 ingestions per second on a modern laptop and achieves a disk compression ratio of at least **8x** over the naive format on realistic data — a number only achievable with both delta-of-delta timestamps and XOR-encoded values working correctly.

3 The Wire Protocol

Plain text, one command per line, terminated by newline. The server responds in kind. These commands are exactly the same as in the simpler version of this project; the compression all lives behind the protocol.

3.1 PUT, GET, AGG, STATS, FLUSH, QUIT

```
PUT metric_name timestamp value
GET metric_name from_timestamp to_timestamp
AGG metric_name from_timestamp to_timestamp bucket_seconds func
STATS metric_name
FLUSH metric_name
QUIT
```

`metric_name` is an identifier (letters, digits, dots, and underscores only). `timestamp` is a 64-bit Unix timestamp in seconds. `value` is a double. `func` is one of `avg`, `min`, `max`, `sum`, `count`.

`PUT` commands for a given metric must arrive in non-decreasing timestamp order; out-of-order inserts are rejected with an error. `GET` returns points in the half-open range [`from`, `to`) as one line per point. `AGG` groups matching points into consecutive buckets and applies the function to each. `STATS` prints a small summary. `FLUSH` forces an immediate flush of the named metric's head block to disk.

4 Architectural Overview

```
          [ TCP connections ]
                  |
                  v
        +---------------------+
        |  Network layer      |  accept connections, parse
        +---------------------+
                  |
                  v
        +---------------------+
        |  Storage engine     |  per-metric head block,
        |                     |  flush, chunk file reader
        +---------------------+
                  |
                  v
        +---------------------+
        |  Compression layer  |  delta-of-delta timestamps,
        |  (Sections 6, 7)    |  XOR-encoded values
        +---------------------+
                  |
                  v
        +---------------------+
        |  Bitstream I/O      |  read/write individual bits
        |  (Section 5)        |  into byte buffers
        +---------------------+
```

The bitstream I/O layer is new compared to a normal database project. Build it first; everything else depends on it.

5 Bitstream I/O

The compression schemes in Sections 6 and 7 produce variable numbers of bits per encoded value. Some timestamps compress to a single bit; some values compress to 13 bits, some to 64. You cannot store these as bytes because they are not byte-aligned. You need a small library for reading and writing individual bits into a byte buffer.

5.1 The Bit Writer

A bit writer holds a growing byte buffer, a current byte being built up, and the number of bits filled in that byte (0 through 7). The interface is one function:

```
void bw_write(BitWriter *bw, uint64_t value, int n_bits);
```

The function appends the low `n_bits` of `value` to the bitstream, most significant bit first. When the current byte fills (the bit count reaches 8), it is appended to the buffer and a new byte is started. When the writer is closed, any partially filled final byte is appended with the remaining bits set to zero.

A typical implementation:

```
void bw_write(BitWriter *bw, uint64_t value, int n_bits) {
    while (n_bits > 0) {
        int free_in_byte = 8 - bw->bits_filled;
        int take = (n_bits < free_in_byte) ? n_bits : free_in_byte;
        int shift = n_bits - take;
        uint64_t chunk = (value >> shift) & ((1ULL << take) - 1);
        bw->current_byte |= chunk << (free_in_byte - take);
        bw->bits_filled += take;
        n_bits -= take;
        if (bw->bits_filled == 8) {
            buffer_append(bw, bw->current_byte);
            bw->current_byte = 0;
            bw->bits_filled = 0;
        }
```

```
    }
}
```

5.2 The Bit Reader

The reverse operation. Holds a byte buffer pointer, a current position, and the number of bits already consumed from the current byte.

```
uint64_t br_read(BitReader *br, int n_bits);
```

Returns the next `n_bits` bits as a 64-bit unsigned integer.

5.3 Test Both Before Anything Else

Before writing a single line of compression code, write a unit test that does the following: create a bit writer, write 1000 random values with random bit widths (1 to 64), close the writer, create a bit reader on the resulting buffer, read back the same 1000 widths in order, and verify each value matches. If this test does not pass, nothing downstream will work, and debugging Gorilla compression with a broken bit writer is a nightmare. Get the bit I/O right first, in isolation.

6 Delta-of-Delta Encoding for Timestamps

The first compression technique is for timestamps. Because points within a metric are usually sampled at regular intervals (every second, every minute, etc.), the difference between consecutive timestamps is almost always the same value. Delta-of-delta encoding takes advantage of this by storing not the timestamps, not even the deltas, but the differences *between* successive deltas, almost all of which are zero.

6.1 The Algorithm

Given a sequence of timestamps $t_1, t_2, t_3, \ldots$:

- Store t_1 in full (64 bits).
- Store $\delta_1 = t_2 - t_1$ in 14 bits (this is the first delta, treated as the "baseline" interval).
- For each subsequent timestamp t_i ($i \geq 3$), compute $\delta_i = t_i - t_{i-1}$, then $D = \delta_i - \delta_{i-1}$. This is the delta-of-delta.
- Encode D using a variable-bit-length scheme based on its magnitude (see below).

For a perfectly regular metric, every D after the first delta is 0, and zero compresses to a single bit. A 10,000-point chunk uses about $64 + 14 + 9999 = 10{,}077$ bits for timestamps, or about 1.26 KB. The naive format would use 80,000 bytes. That is a 60× reduction.

6.2 The Variable-Bit Encoding

The Gorilla paper specifies the following encoding for the delta-of-delta value D:

```
if D == 0:                          write '0'              (1 bit)
elif -63 <= D <= 64:                write '10', then D     (1+2+7 = 9 bits)
elif -255 <= D <= 256:              write '110', then D    (1+3+9 = 12 bits)
elif -2047 <= D <= 2048:            write '1110', then D   (1+4+12 = 16 bits)
else:                               write '1111', then D   (1+4+32 = 36 bits)
```

The prefix tells the decoder how many bits to read for the value itself. The decoder reads bits one at a time until it finds a 0 (or hits four 1s), then reads the appropriate number of bits as a signed integer. Zero deltas-of-deltas (the common case) cost only 1 bit, while rare large jumps cost 36 bits — still better than the 64 bits a raw timestamp would use.

The signed integers in the table are stored using the simplest sign convention: the high bit of the encoded chunk represents the sign. For 7-bit values, the range is -64 to $+63$ stored as a 7-bit two's-complement integer. The Gorilla paper Section 4.1.1 has the exact details and a worked example; refer to it.

7 XOR Encoding for Values

The second compression technique is for the floating-point values themselves. The insight is that two consecutive values in a real-world time series (CPU usage at 45.2 then 45.8) have IEEE 754 representations that differ in only a handful of bits, mostly in the low end of the mantissa. If you XOR the two together, the result has a long run of zeros at the beginning and another long run of zeros at the end, with a small number of meaningful bits in the middle.

7.1 The Algorithm

Given a sequence of values $v_1, v_2, v_3, \ldots$ (each a `double`):

- Store v_1 in full (64 bits, as the raw IEEE 754 bits).
- For each subsequent value v_i ($i \geq 2$), compute the XOR of its bits with the previous value's bits: $X = bits(v_i) \oplus bits(v_{i-1})$.
- Encode X using the variable-bit scheme described next.

Identical consecutive values (very common in metrics that are sampled but unchanged — think a temperature sensor reading the same value every second for an hour) compress to a single bit, since the XOR is zero.

7.2 The Variable-Bit Encoding

The encoding tracks two pieces of state across calls: the number of leading zero bits and the number of trailing zero bits in the previous non-zero XOR (the "previous block"). The encoding rules:

```
if X == 0:                                      write '0'   (1 bit)
elif leading_zeros(X) >= prev_leading
     and trailing_zeros(X) >= prev_trailing:
    # the meaningful bits fit inside the previous block.
    # reuse the previous block's leading-zero count.
    write '10'
    n_meaningful = 64 - prev_leading - prev_trailing
    write the n_meaningful meaningful bits of X      (2 + n_meaningful bits)
else:
    # store new leading-zero count and meaningful-bit count.
    write '11'
    write leading_zeros(X) in 5 bits                  (2 + 5 + 6 + n bits)
    n_meaningful = 64 - leading_zeros(X) - trailing_zeros(X)
    write n_meaningful in 6 bits
    write the n_meaningful meaningful bits of X
    update prev_leading and prev_trailing
```

The first non-zero XOR always takes the `'11'` branch because there is no previous block. After that, runs of similar XOR patterns can stay in the cheaper `'10'` branch.

For typical metric data, identical consecutive values produce the `'0'` branch (1 bit), small drift produces the `'10'` branch (typically around 13 bits), and only large jumps require the full `'11'` branch (about 50 bits worst case). The Gorilla paper reports an average of about 1.37 bytes per value across Facebook's production metrics, and you should be able to reproduce something close to that in your benchmark.

7.3 The Decoder

The decoder mirrors the encoder, maintaining the same prev_leading and prev_trailing state. Read the first bit; if 0, the XOR is 0 and the value is the same as the previous one. Otherwise read another bit; if 0, the meaningful bits use the previous block; if 1, read the new block parameters (5+6 bits) and then the meaningful bits. Reconstruct X, XOR it with the previous value's bits, and emit the result as a `double`.

7.4 Test the Encoder and Decoder in Isolation

As with the bit reader/writer, test the value encoder and decoder before integrating them. Generate a sequence of 1000 doubles with realistic patterns (a slowly drifting random walk is a good simulation of metric data), encode them, decode them, and verify each value comes back exactly equal to the original. Bit-exact equality is required: floating-point comparison can fail in subtle ways if the encoder loses bits.

8 The Storage Engine

This is the same head-block-and-chunks model as the simpler version of this project, but the chunks are now compressed.

8.1 The Head Block

For every metric, the server keeps an in-memory *head block* holding the most recently ingested points as two parallel arrays: timestamps (`int64`) and values (`double`). Writes append to both in $O(1)$ time. The head block has a maximum capacity (a tunable constant; pick something between 1,000 and 10,000 points). When a `PUT` arrives at a full head block, the head block is flushed to a chunk file, the arrays are emptied, and the new point is appended.

The head block stores points uncompressed; compression only happens at flush time. This keeps the in-memory data structure simple and the ingestion path fast.

8.2 The Metric Registry

A hash map from metric name to head block. At server startup, the data directory is scanned: every subdirectory corresponds to a metric, and the presence of any chunk file is enough to register it with an empty head block.

8.3 Flushing

A flush takes the points currently in a metric's head block and writes them as a single compressed chunk file in the per-metric subdirectory. Chunks are named after their first timestamp:

```
./data/cpu.usage/1728000000.chunk
./data/cpu.usage/1728010000.chunk
```

Use temporary-file-and-rename for atomic flush: write to `.tmp`, `fsync`, `rename()`.

8.4 Query Execution

A `GET` or `AGG` needs every point in a time range, possibly spread across multiple chunk files plus the head block:

1. List the chunk files in the metric's directory. Sort by the timestamp embedded in each filename.
2. For each chunk file: read its header, check whether its timestamp range overlaps the query range. If not, skip. If yes, decompress the chunk and keep points inside the query range.
3. Merge in the head block's points (if any are in range).
4. Return the points in ascending timestamp order. For `AGG`, group into buckets and reduce.

9 The Chunk File Format

A chunk file is a binary file with a header followed by two compressed bitstreams (timestamps and values) and a footer. All multibyte integers are little-endian.

```
Offset  Size       Field               Description
------  ---------  -----------------   ------------------------
  0     4          magic               ASCII bytes 'T','S','D','B'
  4     4          version             uint32, must be 2
  8     4          point_count (N)     uint32
 12     8          first_timestamp     int64
 20     8          last_timestamp      int64
 28     4          ts_bitstream_len    uint32, length in bytes
 32     4          val_bitstream_len   uint32, length in bytes
 36     ...        timestamp bitstream (Section 6)
 ...    ...        value bitstream      (Section 7)
 ...    8          crc64               over everything above
```

Both bitstreams use the bit writer from Section 5. They are byte-aligned at their boundaries (the timestamp bitstream is padded to a whole number of bytes before the value bitstream begins), so the reader can locate them via the `ts_bitstream_len` field.

The CRC at the end protects the entire file against corruption; on read, verify it before trusting any of

the data.

9.1 Why This Compresses Well

With N points, the naive format uses $16N$ bytes. The compressed format uses 36 bytes of header plus 8 bytes of CRC plus the timestamp bitstream (typically about 1.3 bits per point for regular metrics) plus the value bitstream (typically about 13 bits per point for slowly drifting metrics). For $N = 10{,}000$ that is roughly $44 + 1625 + 16250 \approx 17.9$ KB, compared to 160 KB naive — close to a 9× ratio. Real metrics often do better still.

10 Phase 1 — Network Layer and In-Memory Storage

The goal of Phase 1 is a working TCP server that handles `PUT`, `GET`, `STATS`, and `QUIT` with all data in memory. No compression, no disk.

Build:

- A TCP server with one thread per client.
- A line-based protocol parser.
- The metric registry (hash map of metric name to head block).
- The head block: parallel timestamp and value arrays with append, range scan, and size accounting.
- Handlers for `PUT` (with monotonicity check), `GET` (linear scan), `STATS`, and `QUIT`.

At the end of Phase 1 you can insert points and read them back, but nothing reaches disk.

11 Phase 2 — Compression and the Chunk File Format

The goal of Phase 2 is the bit-level compression and the disk format. **Read the Gorilla paper before starting this phase.**

Build, in this order:

- The bit writer and bit reader (Section 5). Test in isolation with the round-trip test described in Section 5.3.
- The delta-of-delta timestamp encoder and decoder (Section 6). Test in isolation: feed in a regular sequence (e.g., timestamps every 10 seconds for 1000 points), encode, decode, verify exact recovery.
- The XOR value encoder and decoder (Section 7). Test in isolation with a slowly drifting random walk of 1000 doubles.
- The chunk file writer: given a head block, produce a compressed chunk file using the format in Section 9. Use temp-and-rename.
- The chunk file reader: open a chunk, decompress both bitstreams, return iterators of (timestamp, value) pairs.
- Automatic flushing on head block overflow.
- The `FLUSH` command.
- Updated `GET` that reads from chunks plus the head block, in order.
- Startup metric discovery: scan the data directory.

At the end of Phase 2, the example session in Section 2 should work end to end. Insert points, force a flush, see the compressed file appear, read the points back, verify they match what was inserted bit-for-bit.

11.1 Debugging Tip

The most common compression bug is an off-by-one in the variable-bit encoding (writing 7 bits when you meant 9, or vice versa). When this happens, the first few values decode correctly and then everything afterward is garbage. If you see this symptom, test the encoder and decoder in isolation again with hand-crafted inputs whose bit layouts you can compute on paper.

12 Phase 3 — Aggregation, Benchmark, and Compression Verification

The goal of Phase 3 is the aggregation query and the benchmark report.

Build:

- The `AGG` command, supporting `avg`, `min`, `max`, `sum`, `count`. Implement as a single pass over the merged head-block-plus-chunks output, grouping into buckets.
- A benchmark program that:
  - Generates 500,000 synthetic points across at least 10 metrics. Use timestamps at regular 1-second intervals (so delta-of-delta works) and values that follow a slowly drifting random walk (so XOR works).
  - Inserts every point through `PUT`. Measures wall-clock time. Reports points per second.
  - Forces a flush of every metric. Measures the total bytes on disk in the data directory.
  - Generates the same points in a separate file using the naive 16-bytes-per-point format.
  - Reports the compression ratio: naive bytes divided by actual bytes on disk.
- A handful of `GET` and `AGG` queries against the loaded data, with measured latencies, to demonstrate read correctness at scale.

A successful submission sustains at least 50,000 ingestions per second and achieves a compression ratio of at least **8x**. If your ratio is significantly lower, you have a bug somewhere in the encoding (probably the XOR encoder is falling into the expensive branch too often). If your ratio is much higher than 10x on this benchmark, your data is too regular — vary the random walk more.

The design document must include the benchmark output as a table, plus a paragraph reasoning about your specific compression ratio and how it compares to the theoretical best case (about 11–12x for ideally compressible data).

13 Deliverables and Submission

A single zip archive containing the source tree, `Makefile`, `README.md`, and:

- A `design.pdf` of at most 10 pages describing the three architectural layers, the chunk file format, the bit-level encoders for both timestamps and values (with worked examples of how a representative input is encoded bit by bit), and the benchmark results.
- A `benchmark/` directory with the benchmark and your final results.
- A `tests/` directory with isolated unit tests for the bit reader/writer, the timestamp encoder/decoder, the value encoder/decoder, and the end-to-end chunk round-trip.

Archive naming: `Group[Number]_Project02_TSDB.zip`

Demonstration: 25-minute live demonstration. The instructor will, in addition to the usual walk-through and benchmark run, request that you decode a specific bit pattern from one of your chunk files by hand on the whiteboard — proving you understand what the bytes mean. All group members must be present.

14 Bonus Opportunities

Up to 15 percentage points total. Each independent.

14.1 Write-Ahead Log and Crash Recovery (up to 10%)

Add a per-metric WAL written before each `PUT`. On startup, replay it to rebuild the head block. After a successful flush, truncate the WAL. Demonstrate that killing the server mid-ingestion and restarting loses no acknowledged `PUT`s.

14.2 Retention Policies (up to 5%)

A configuration option specifying per-metric maximum age. A background task running every minute deletes chunk files whose `last_timestamp` is older than the retention horizon. Discuss the interaction with in-flight queries.

14.3 Downsampling (up to 5%)

Periodically produce a second copy of old data at a coarser resolution: for chunks older than one hour, generate a parallel set of chunks containing one point per minute (the average over each minute), stored in a sibling directory. Aggregation queries that span long ranges can read the coarse data instead, dramatically reducing query cost. Describe the dispatch strategy in the design document.

15 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing time-series database or storage engine as a library or component (InfluxDB, Prometheus, TimescaleDB, Beringei, RocksDB, LevelDB, SQLite, and their bindings).
- Use of any existing binary serialization or compression library (Protobuf, Cap'n Proto, FlatBuffers, snappy, zstd, gorilla-style libraries from any language). The bit I/O and both encoding schemes must be hand-written.
- Use of text-based persistence (JSON, CSV) as the chunk format.
- Storing each data point as its own file, or storing metrics as SQL tables.
- Skipping the isolated round-trip tests for the bit I/O, the timestamp encoder, and the value encoder. These tests are required and must be present in your `tests/` directory.
- Use of Python at any layer, including test harnesses and benchmark drivers.
- Submitting code whose commit history shows large, infrequent commits.

16 Required Reading

1. Pelkonen, T. et al. "Gorilla: A Fast, Scalable, In-Memory Time Series Database." *Proceedings of the VLDB Endowment*, 8(12), 2015. The Facebook paper that introduced both delta-of-delta timestamp encoding and XOR-based float encoding. Sections 4.1.1 (timestamps) and 4.1.2 (values) describe exactly the algorithms you will implement, with worked examples and bit-level diagrams. **Required reading before Phase 2**; the manual you are reading now is a summary, but the paper is the authoritative specification.
2. Reinartz, F. "Writing a Time Series Database from Scratch." Prometheus blog, 2017. A high-level architectural description of how a TSDB is structured. The sections on head blocks and chunk files describe the storage model used in this project.
3. Kleppmann, M. *Designing Data-Intensive Applications*, Chapter 3, section on log-structured storage and SSTables. Background on the general pattern this project specializes for time series.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM